

Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优

原创 已于 2025-07-25 10:50:51 修改 · 2k 阅读 · 58 点赞 · 50 收藏 · CC 4.0 BY-SA 版权

文章标签： [#unity](#) [#游戏引擎](#) [#架构](#) [#c#](#)

Unity GC 系列教程第一篇：GC 基础概念与工作原理

Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具

Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）

Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优

Unity GC 系列教程第五篇：高级 GC 内核

欢迎来到 [Unity](#) GC 系列教程的第四篇！在上一篇文章中，我们深入探讨了如何通过优化装箱、字符串、集合、[对象池](#) 以及常见的 Unity API 调用来显著减少 **GC Alloc**。现在，我们将继续深入，探讨一些更高级的 GC Alloc 优化技巧，并首次触及 GC 自身的调优，包括 Unity 引入的 **Incremental GC**。

本篇内容将帮助你进一步精炼代码，减少那些更隐蔽的内存分配，并了解如何配置 Unity 的 GC 行为，以实现更流畅的游戏体验。

4.1 深入理解和优化 **foreach** 循环

foreach 循环因其简洁的语法而广受欢迎。然而，对于某些类型的集合，它可能会在后台产生临时的 **枚举器 (Enumerator)** 对象，从而导致 GC Alloc。

原理：

当你在 C# 中使用 `foreach` 循环遍历一个实现了 `IEnumerable` 接口的类型时，编译器会在内部调用该类型的 `GetEnumerator()` 方法来获取一个枚举器。

- 如果 `GetEnumerator()` 方法返回的是一个 **值类型 (struct)** 枚举器，那么这个枚举器会在栈上分配，不会产生 GC Alloc。例如，`List<T>` 和数组 (`T[]`) 的 `GetEnumerator()` 方法都返回值类型枚举器，所以直接对 `List<T>` 和 `T[]` 使用 `foreach` 是会产生 GC Alloc 的。
- 如果 `GetEnumerator()` 方法返回的是一个 **引用类型 (class)** 枚举器，那么每次 `foreach` 循环都会在堆上分配一个新的枚举器对象，从而导致 GC Alloc。

常见场景：

以下是一些常见或需要注意的 `foreach` 产生 GC Alloc 的情况：

1. **非泛型集合**：如 `System.Collections.ArrayList`、`System.Collections.Hashtable`。它们的 `GetEnumerator()` 方法返回引用类型 (`IEnumerator` 接口)。

C#

AI运行代码复制

```
1 using System.Collections;
2 using UnityEngine;
3
4 public class ForeachBadExample : MonoBehaviour
5 {
6     void Start()
7     {
8         ArrayList myArrayList = new ArrayList { 1, 2, 3 };
9         foreach (int item in myArrayList) // GC Alloc: ArrayList.GetEnumerator() 返回引用类型
10
11     }
```

展开

2. 某些 Unity 提供的集合或 API 的返回值：虽然大部分 Unity 新 API 都尽量避免 GC Alloc，但一些老旧或特定设计的 API 仍然可能返回会产生 GC Alloc 的可枚举类型。
3. 自定义可枚举类型：如果你自己实现 IEnumerable<T> 或 IEnumerable 接口，并且你的 GetEnumerator() 返回的是一个 class 类型，或者在内部有装箱行为，那么 foreach 就会产生 GC Alloc。

好代码示例（避免 foreach 的 GC Alloc）：

- 优先使用 for 循环：对于 List<T> 和数组 (T[])，使用 for 循环总是最安全、最直接、且无 GC Alloc 的方式。它直接通过索引访问元素。

C#

AI运行代码复制

```
1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class ForeachGoodExample : MonoBehaviour
5 {
6     private List<int> myIntList = new List<int> { 1, 2, 3 };
7     private int[] myIntArray = new int[] { 4, 5, 6 };
8
9     void Start()
```

展开

- 自定义值类型迭代器：如果你需要实现自己的可枚举类型，并希望它在 foreach 中不产生 GC Alloc，你需要确保其 GetEnumerator() 方法返回一个 struct 类型的枚举器，并且这个 struct 枚举器本身不包含任何引用类型字段，或者其内部操作不会产生额外的 GC Alloc。

C#

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 // 一个自定义的、无 GC Alloc 的可枚举 struct
6 public struct MyOptimizedCollection : IEnumerable<int>
7 {
8     private int[] _data;
9 }
```

展开 ∨

实现自定义值类型迭代器相对复杂，通常只在性能极度敏感且需要自定义枚举行为的场景下考虑。

总结：对于 `foreach` 循环，最简单的优化是：

- 对 `List<T>` 和数组 (`T[]`)，直接使用 `foreach` 通常是安全的，因为它们的枚举器是值类型。
- 对于其他集合，特别是非泛型集合或你不确定其枚举器类型的集合，**优先使用 `for` 循环配合索引访问**。
- 如果你正在编写自定义集合，并且希望它能与 `foreach` 无缝衔接且无 GC Alloc，请确保你的 `GetEnumerator()` 方法返回一个 `struct`。

4.2 LINQ 的替代方案

LINQ (Language Integrated Query) 以其优雅和强大的查询能力而闻名。然而，正如我们在第二篇中提到的，**大多数 LINQ 查询都会在内部创建临时的集合或枚举器，导致大量的 GC Alloc**。在游戏的热路径中（如 `Update` 或 `FixedUpdate`），应尽可能避免使用 LINQ。

原理回顾：LINQ 扩展方法通常返回 `IEnumerable<T>` 或其他内部迭代器，它们通常是引用类型。当链式调用多个 LINQ 方法时，每个中间结果都可能是一个临时的引用类型对象。

坏代码示例：

C#

AI运行代码 复制

```
1 using System.Linq;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class LinqBadExample : MonoBehaviour
6 {
7     private List<Enemy> _allEnemies = new List<Enemy>();
8
9     void Update()
```

展开

上述代码中，即使不考虑 `foreach` 可能产生的枚举器问题，`Where` 每次链式调用和 `ToList` 都会产生 GC Alloc。

好代码示例（手动实现 LINQ 逻辑）：

手动编写循环逻辑，使用预分配的集合来存储结果，是避免 LINQ 带来的 GC Alloc 的最佳实践。

C#

AI运行代码 复制

```
1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class LinqGoodExample : MonoBehaviour
5 {
6     private List<Enemy> _allEnemies = new List<Enemy>();
7
```

```
'
8 // 缓存一个 List 来存储筛选结果，避免每次都 new
9 private List<Enemy> _cachedTargetEnemies = new List<Enemy>();
```

展开 ∨

通过手动循环，我们避免了 LINQ 内部产生的临时迭代器和集合，只用了一个预分配并重用的 `_cachedTargetEnemies` List。

总结：

- 在性能敏感的热路径中，坚决避免使用 LINQ。
- 手动编写循环和条件判断来替代 LINQ 表达式。
- 使用预分配的、可重用的集合来存储中间和最终结果。
- LINQ 并非一无是处，在非性能敏感的初始化 代码、编辑器工具或一次性脚本中，它仍然是提高代码可读性和开发效率的利器。关键在于“何时何地”使用。

4.3 避免匿名函数和 Lambda 表达式捕获变量 (Closures)

闭包 (Closures) 是 C# 中一个强大但容易产生隐性 GC Alloc 的特性。当一个 匿名方法 或 Lambda 表达式 引用了其外部作用域的局部变量时，编译器为了让这些变量能在外部生命周期之外仍然可用，会为这些被捕获的变量生成一个匿名类。这个匿名类的实例就是在堆上分配的，从而产生了 GC Alloc。

原理回顾：编译器会将捕获的局部变量“提升”为这个匿名类的字段，并在堆上创建一个该类的实例。

坏代码示例：

C#

```
1 using UnityEngine;
2 using System;
3 using System.Collections.Generic;
4
5 public class ClosureBadExample : MonoBehaviour
6 {
7     void Start()
8     {
9         // 示例1: 单次捕获
```

展开

在示例1中，`myAction` 只是一个单独的 GC Alloc。但在示例2的循环中，**每次迭代都会创建一个新的闭包对象**，导致 5 次 GC Alloc，即便这些对象在循环结束后可能很快变为垃圾。示例3也揭示了闭包在事件订阅/取消订阅中的陷阱。

好代码示例（避免闭包 GC Alloc）：

- **避免捕获外部变量：** 如果可能，将需要捕获的变量作为参数传递给一个普通方法，或者确保 Lambda 表达式不需要访问外部局部变量。

C#

```
1 using UnityEngine;
2 using System;
3 using System.Collections.Generic;
4
5 public class ClosureGoodExample : MonoBehaviour
6 {
7     void Start()
8     {
9
```


展开 ▼

在循环中捕获变量时，通过在循环体内部声明一个新的局部变量 (`capturedIndex = i;`) 来创建变量的**副本**，这确保了每次迭代的闭包捕获的是不同的、独立的变量副本。

总结：

- 在性能敏感的代码中，**尽量避免使用捕获外部变量的 Lambda 表达式。**
- 如果必须捕获，请仔细考虑捕获的生命周期，确保它只发生一次，而不是在循环或频繁调用的方法中重复发生。
- 对于事件订阅和取消订阅，如果 Lambda 表达式捕获了变量，那么每次生成的 Lambda 实例都是不同的，你将无法通过重新定义相同的 Lambda 来取消订阅。你需要缓存委托实例才能正确取消订阅。

4.4 GC 调优：Incremental GC (增量式 GC)

在 Unity 2019.3 及更高版本中，Unity 引入了对 **Incremental GC (增量式 GC)** 的支持。这是 Unity GC 性能优化的一个里程碑，因为它极大地改善了传统的阻塞式 GC 导致的卡顿问题。

4.4.1 什么是 Incremental GC?

传统阻塞式 GC 的主要问题是 **Stop-The-World (STW)** 暂停：GC 运行时会暂停所有应用程序线程，直到回收工作完成。这在游戏这种实时应用中会导致明显的卡顿。

Incremental GC 的核心思想是：**将 GC 的工作分解为多个小的、非阻塞的增量步骤，在多帧中逐步完成，而不是一次性完成所有工作。** 这样，每次 GC 暂停的时间就会大大缩短，通常只有几毫秒甚至更短，从而减少或消除玩家感知到的卡顿。

想象一下你有一堆脏衣服要洗。阻塞式 GC 就像你一次性把所有衣服都洗完（然后你就没法做其他事情了）。而增量式 GC 就像你分批洗，每次只洗几件，中间你可以继续做其他事情，虽然整个洗衣过程的总时间可能差不多，但你不会感到“卡住”。

4.4.1 Incremental GC 的工作原理 (简化版)

增量 GC 通常基于 **三色标记法 (Tri-color Marking)**：

1. **白对象 (White Objects)**：未被 GC 访问到的对象，初始时所有对象都是白色。它们是潜在的垃圾。
2. **灰对象 (Gray Objects)**：已经被 GC 访问到，但其引用的对象还没有被全部遍历。
3. **黑对象 (Black Objects)**：已经被 GC 访问到，并且其引用的所有对象也已经被遍历。它们是存活的对象。

Incremental GC 的流程大致如下：

- GC 从 GC 根开始，逐步将对象从白色标记为灰色，再从灰色标记为黑色。
- 这个标记过程可以被应用程序中断，GC 线程可以暂停，让游戏线程继续运行。
- 为了保证在 GC 暂停期间对象引用关系不被破坏，增量式 GC 需要引入 **写屏障 (Write Barrier)** 机制。当应用程序线程修改一个对象的引用时（例如，将一个黑色对象引用到一个白色对象），写屏障会确保这个白色对象被重新标记为灰色，从而确保它不会在 GC 清理阶段被错误回收。
- 当所有对象都被标记完毕后，GC 会进行一次短暂的 **STW 阶段 (Mark-Sweep 的清扫阶段)** 来实际回收白色对象。由于标记工作已经分散到多帧中，这个最终的 STW 阶段通常非常短。

4.4.3 如何开启和配置 Incremental GC?

Incremental GC 在 Unity 2019.3 及更高版本中是默认启用的，但你可以在 **Project Settings -> Player -> Other Settings -> Configuration -> Use Incremental GC** 中手动控制它。

(这是一个示意图，实际界面可能略有不同。注意箭头指向的 *Use Incremental GC*)

建议：在绝大多数情况下，**你应该启用 Incremental GC**。它能显著改善游戏的流畅性，减少玩家感知到的卡顿。

何时可能需要关闭？

极少数情况下，如果你的游戏有非常独特的内存模式，或者你正在调试一些与 GC 内部机制相关的复杂问题，可能会考虑暂时关闭它。但对于绝大多数游戏，启用 Incremental GC 是明确的性能提升。

4.4.4 Incremental GC 如何减少 GC Pause？

- **分散工作负载**：将一次性的大量 GC 工作分散到多帧中，每次只做一点点，从而避免了长时间的阻塞。
- **短暂停顿**：每次GC步骤的暂停时间非常短，通常在毫秒级以下，使得它们对帧率的影响微乎其微，甚至在 Profiler 中都难以察觉为明显的“尖峰”。
- **更好的响应性**：应用程序线程可以更快地响应用户输入和更新游戏逻辑。

4.4.5 Incremental GC 的适用场景和局限性

- **适用场景**：所有需要高帧率和流畅体验的实时游戏。对于大型开放世界、快节奏动作游戏、VR/AR 应用等尤其重要。
- **局限性**：
 - **并非完全消除 GC Pause**：虽然每次暂停时间极短，但仍有短暂的 STW 阶段。
 - **不减少 GC Alloc**：Incremental GC 优化的是 GC 的“执行方式”，而不是“何时执行”。如果你的游戏仍然频繁地产生大量 GC Alloc，那么即使有了 Incremental GC，GC 也会更频繁地被触发，虽然每次暂停短，但总的 GC 时间和 CPU 开销仍然存在。因此，**减少 GC Alloc 仍然是首要任务**。
 - **写屏障开销**：写屏障会带来微量的额外 CPU 开销。但在绝大多数情况下，这种开销远低于阻塞式 GC 带来的卡顿影响。

重要提示：启用 Incremental GC 并不意味着你可以忽略 GC Alloc 优化。相反，这两者是相辅相成的。**GC Alloc 优化是治本，Incremental GC 是治标（改善症状）**。只有当你的 GC Alloc 量被控制在较低水平时，Incremental GC 才能发挥最大效果，确保那些短促的 GC 步骤不会因

为过于频繁而累积成新的问题。

4.5 手动触发 GC (`System.GC.Collect()`)

C# 提供了手动触发 GC 的方法： `System.GC.Collect()` 。

作用：强制运行时立即执行一次垃圾回收。它有一个重载 `System.GC.Collect(int generation)` ，可以指定回收哪个代（通常是最高代）。

何时使用？

在游戏开发中，通常不建议频繁或盲目地手动触发 GC。因为 GC 的执行时机是由运行时根据内存压力和内部启发式算法 来决定的，它通常知道什么时候是最佳时机。强制触发 GC 可能会在不合时宜的时候（例如在激烈战斗中）导致意外的卡顿。

然而，在某些特定场景下，手动触发 GC 可能是有用的：

- **加载屏幕/场景切换：**在游戏加载新场景或在加载屏幕期间，CPU 通常没有其他重要的游戏逻辑在执行。这是一个理想的时机来手动触发一次 GC，清理旧场景遗留的垃圾，从而确保新场景开始时有一个干净的内存环境。

C#

AI运行代码 复制

```
1 IEnumerator LoadSceneAsync(string sceneName)
2 {
3     // 卸载当前场景资源（如果需要）
4     yield return Resources.UnloadUnusedAssets(); // 这个也会触发一次GC
5
6     // 在加载新场景之前，强制执行一次GC
7     System.GC.Collect();
8     Debug.Log("Forced GC collection before scene load.");
9 }
```

展开 ▼

- **内存压力测试**：在开发和调试阶段，手动触发 GC 可以帮助你模拟 GC 行为，检查内存泄漏或 GC 峰值是否按预期出现。

注意事项：

- **不要在游戏循环 (如 `Update()`) 中频繁调用 `System.GC.Collect()`**。这会直接导致严重的卡顿。
- `System.GC.Collect()` 会导致一次 **阻塞式 GC**，即使启用了 Incremental GC，它也会强制执行一次完整的 STW 回收。因此，使用时要特别谨慎，并确保在用户不会察觉到卡顿的非关键时刻进行。
- `Resources.UnloadUnusedAssets()` 也会触发一次 GC，因为它需要清理那些不再被引用的资源。通常在场景切换时配合使用。

4.6 理解堆内存 (Managed Heap)

在 GC 优化中，理解 **托管堆 (Managed Heap)** 是至关重要的。托管堆是 GC 管理所有引用类型对象的地方。

- **堆的增长**：当你的程序不断创建新对象，而 GC 还没来得及回收时，托管堆会不断增长。如果堆的增长超出了某个阈值，就会触发 GC。
- **堆的收缩**：GC 回收内存后，堆的大小可能会收缩，但通常不会立即收缩到最小。这是因为操作系统分配内存是按页进行的，GC 也不会频繁地将内存归还给操作系统。
- **虚拟内存 (Virtual Memory)**：操作系统会为每个进程分配一块虚拟内存。当程序申请内存时，实际上是向操作系统申请虚拟内存。只有当这些虚拟内存真正被使用时，操作系统才会将其映射到物理内存上。GC 优化主要关注的是托管堆在虚拟内存中的使用量，以及它实际消耗的物理内存。

Profiler 中的堆信息：

在 Unity Profiler 的 Memory 模块中，你可以看到详细的堆内存使用情况：

- **Total Reserved**：程序保留的虚拟内存总量。

- **Total Allocated**：实际从操作系统分配的物理内存总量。
- **Total Used**：实际被存活对象使用的内存量。
- **GC Allocated In Frame**：当前帧 GC 的总分配量（与 CPU Usage 模块中的 `GC.Alloc` 对应）。

(这是一个示意图，实际界面可能略有不同。注意箭头指向的 `Managed Heap` 信息)

优化目标：

- **降低“Total Used”的峰值**：通过减少不必要的对象创建和及时释放资源，使存活对象占据的内存尽可能小。
- **减少“GC Allocated In Frame”**：这是我们本系列教程的主要目标，通过之前和本篇介绍的优化技巧来达成。
- **观察“Total Allocated”的变化**：如果这个值持续增长，可能意味着存在内存泄漏（即使 GC 已经回收了大部分，但某些引用阻止了关键对象的回收）。

4.7 总结 GC Alloc 优化策略

至此，我们已经介绍了大量的 GC Alloc 优化技巧。我们可以将它们归纳为几个核心策略：

1. **重用对象 (Object Pooling)**：这是最重要的策略，尤其适用于频繁创建和销毁的同类型对象。
2. **缓存 (Caching)**：缓存那些在多次调用中结果不变的对象或组件引用。
3. **使用 `NonAlloc` 变体**：对于 Unity API，优先使用 `XXXNonAlloc` 或接受 `List<T>` 参数的方法。
4. **避免装箱 (Boxing)**：使用泛型集合、`StringBuilder`、泛型方法，避免值类型到 `object` 的隐式转换。
5. **避免不必要的字符串操作**：频繁拼接使用 `StringBuilder`，避免 `Substring`、`Split` 等操作，考虑 `Span<char>`。

6. **优化集合使用**：预分配容量、重用集合（`Clear()`）、避免 `ToArray()` / `ToList()`。

7. **谨慎使用 `foreach`**：对于非 `List<T>` 和数组的集合，考虑 `for` 循环。

8. **警惕闭包 (Closures)**：避免在热路径中创建捕获外部变量的 Lambda 表达式。

9. **避免 LINQ**：在性能敏感代码中，手动实现 LINQ 逻辑。

优化流程：

1. **测试和测量**：永远不要在没有测量的情况下优化。使用 **Unity Profiler** 定位 GC Alloc 的具体来源和数量。

2. **找出热点**：优先优化那些在 `Update`、`FixedUpdate` 或其他高频循环中产生大量 GC Alloc 的代码。

3. **从小处着手**：从最容易改变且效果最明显的点开始（例如，缓存 `WaitForSeconds`、替换 `string + string` 为 `StringBuilder`）。

4. **逐步推进**：对于复杂的优化（如对象池），逐步引入并测试。

5. **持续监控**：优化后，继续使用 Profiler 验证效果，确保没有引入新的问题。

总结

在本篇教程中，我们继续深入了 **GC Alloc 的优化技巧**：

- 我们详细探讨了 `foreach` 循环可能带来的 GC Alloc 问题，并提供了使用 `for` 循环和自定义值类型迭代器的优化方案。
- 我们再次强调了在性能敏感区域 **避免使用 LINQ**，并展示了如何手动实现相同的逻辑以消除内存分配。
- 我们深入分析了 **匿名函数和 Lambda 表达式捕获变量（闭包）** 导致的隐性 GC Alloc，并提供了避免或优化这些场景的方法。

最重要的是，我们首次接触了 **GC 自身的调优**：

- 详细介绍了 **Incremental GC (增量式 GC)** 的概念、原理、开启方式及其如何显著减少 GC 暂停时间，提升游戏流畅性。我们强调了 Incremental GC 虽然改善了症状，但减少 GC Alloc 仍是治本之道。
- 讨论了 **手动触发 GC (`System.GC.Collect()`)** 的适用场景（如加载屏幕）和注意事项，强调了其潜在的阻塞风险。
- 最后，我们简要回顾了 **托管堆** 的概念，并强调了在 Profiler 中监控堆内存的重要性。

至此，你已经掌握了大量的 GC Alloc 优化技巧以及 GC 调优的基础知识。在下一篇也是本系列的最后一篇中，我们将讨论一些更高级的 GC 相关话题，包括 C# Job System, Burst Compiler 以及 Native Container 等 Unity 现代高性能技术如何从根本上解决 GC 问题，并展望未来。

你是否觉得这些优化技巧对你有所启发？有没有哪些是你目前在项目中就想尝试的？

[Unity GC 系列教程第一篇：GC 基础概念与工作原理](#)

[Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具](#)

[Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）](#)

[Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优](#)

[Unity GC 系列教程第五篇：高级 GC 内核](#)

关于我们 招贤纳士 商务合作 寻求报道  400-660-0108  kefu@csdn.net  在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心

家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照

©1999-2025北京创新乐知网络技术有限公司